

Proyecto 3: Algoritmos On-line y Analisis Competitivo

Jonathan Diaz | Carne: 23837 | Universidad del Valle de Guatemala

31 de mayo de 2026

Curso: Analisis de Algoritmos

Proyecto: Algoritmos On-line y analisis competitivo

Estudiante: Jonathan Diaz

Carne: 23837

Repositorio publico: <https://github.com/Jonialen/proyecto3>

Video de YouTube: [pegar aqui el enlace no listado]

Espacio para video:

[pegar aqui el enlace del video no listado de YouTube]

Objetivo

Aplicar los conceptos de algoritmos on-line, metodo del potencial y analisis competitivo mediante una simulacion reproducible.

El proyecto demuestra:

- Como cambia el costo de una lista segun su politica de reorganizacion.
- Como se mide el costo real de accesos e intercambios adyacentes.
- Como se calcula una funcion de potencial basada en inversiones.
- Como se compara un algoritmo on-line contra una referencia offline.
- Como una secuencia puede favorecer o castigar una heuristica.

Problema estudiado

El problema utilizado es el problema del diccionario o actualizacion de listas. Se mantiene una lista lineal de elementos y se procesa una secuencia de solicitudes.

La version implementada se concentra en accesos, aunque el codigo base tambien incluye insercion y eliminacion para los algoritmos on-line.

El costo usa posiciones indexadas desde 1:

Operacion	Costo
Acceder al elemento en posicion i	i
Eliminar el elemento en posicion i	i
Insertar en una lista de tamano n	$n + 1$
Intercambio adyacente	1

Si `MoveToFront` accede a un elemento en posicion k , primero paga el acceso y luego mueve el elemento al inicio mediante $k - 1$ intercambios. Por tanto, su costo real en ese evento es:

$$t_i = k + (k - 1) = 2k - 1$$

Algoritmos implementados

StaticList

StaticList mantiene el orden original de la lista. Sirve como linea base simple porque no invierte costo en reorganizar la estructura.

MoveToFront

MoveToFront mueve al frente cada elemento accedido. La idea es explotar localidad de referencia: si un elemento fue consultado recientemente, es probable que vuelva a consultarse pronto.

Segun el analisis visto en el material del curso, **MoveToFront** es un algoritmo on-line estrictamente 4-competitivo para este problema. Esto significa que su costo queda acotado por un multiplo constante del costo de una referencia optima offline.

Si el elemento solicitado esta en posicion k para **MoveToFront** y en posicion i para la referencia, el material obtiene la cota:

$$a_i \leq 4i$$

Como i es el costo de acceso en la referencia, esta desigualdad conecta directamente el costo amortizado de **MoveToFront** con el costo del algoritmo comparado.

Bit

Bit asigna un bit a cada elemento. Antes de acceder a un elemento, complementa su bit. Si el bit queda en 1, mueve el elemento al frente; si queda en 0, lo deja en su posicion.

El material propone comparar **Bit** contra una referencia optima usando una forma general con factor multiplicativo y termino aditivo:

$$bit \leq k \cdot opt + \alpha$$

El termino α permite capturar diferencias iniciales o costos constantes que no dependen directamente del tamano de la secuencia. En este proyecto no se demuestra formalmente una cota para **Bit**; se implementa y se mide experimentalmente contra la referencia offline.

OfflineOptimalReference

OfflineOptimalReference representa una referencia offline exacta para secuencias pequenas de solo accesos. Conoce la secuencia completa y usa programacion dinamica sobre todas las permutaciones posibles de la lista.

La referencia offline esta limitada porque el numero de estados crece factorialmente:

$$|S| = n!$$

Por esa razon se usa solo para listas pequenas. Esta restriccion mantiene el foco pedagogico en el analisis competitivo.

Metodo del potencial

La funcion de potencial compara dos configuraciones de lista. En este proyecto, el potencial es el numero de inversiones entre la lista del algoritmo analizado y la lista de referencia.

Una inversion es un par (x, y) tal que x aparece antes que y en una lista, pero despues en la otra.

La funcion de potencial se define como:

$$\Phi(L_{alg}, L_{ref}) = \#\{(x, y) : x \prec_{alg} y \wedge y \prec_{ref} x\}$$

El costo amortizado de un evento es:

$$a_i = t_i + \Delta\Phi$$

Donde a_i es el costo amortizado, t_i es el costo real y $\Delta\Phi = \Phi_i - \Phi_{i-1}$ es el cambio de potencial.

Analisis competitivo

El analisis competitivo compara un algoritmo on-line contra una referencia optima offline. Un algoritmo **alg** es k -competitivo si existe una constante α tal que, para toda secuencia de entrada σ :

$$alg(\sigma) \leq k \cdot opt(\sigma) + \alpha$$

En los experimentos se reporta una razon competitiva empirica:

$$r = \frac{costo(alg)}{costo(ref)}$$

Esta razon no prueba formalmente la competitividad del algoritmo, pero permite observar su comportamiento en secuencias concretas.

El material tambien presenta condiciones suficientes para usar el metodo del potencial en analisis competitivo. Para cada evento e_i , se busca que el costo amortizado del algoritmo quede acotado por el costo de la referencia:

$$alg_i \leq k \cdot opt_i$$

Ademas, la funcion de potencial debe estar acotada inferiormente por una constante independiente de la secuencia:

$$\Phi_i \geq b$$

En el caso de la funcion usada en este proyecto, Φ cuenta inversiones. Por definicion, nunca hay un numero negativo de inversiones, entonces se cumple:

$$\Phi_i \geq 0$$

Implementacion

El proyecto esta implementado en Python y usa `uv` para gestionar entorno, dependencias y ejecucion.

Archivo	Responsabilidad
src/analysis/costs.py	Modelo de costos.
src/analysis/potential.py	Calculo de inversiones, potencial y costo amortizado.
src/algorithms/static_list.py	Lista estatica.
src/algorithms/move_to_front.py	Heuristica MoveToFront.
src/algorithms/bit.py	Algoritmo Bit.
src/algorithms/offline_optimal.py	Referencia offline exacta para casos pequenos.
src/experiments/loader.py	Carga de experimentos JSON.
src/experiments/report.py	Construccion de reportes.
src/main.py	CLI del proyecto.

Comando base para ejecutar pruebas:

```
uv run pytest
```

Comando para ejecutar un experimento con referencia offline:

```
uv run python -m src.main data/examples/repeated_accesses.json --bit-seed 1 --include-offline --reference
```

Comando para imprimir las respuestas solicitadas por el enunciado oficial:

```
uv run python -m src.main --project-activities
```

Actividades solicitadas

El enunciado oficial pide calcular costos de acceso usando MTF para la configuracion inicial:

$$[0, 1, 2, 3, 4]$$

En esta seccion, el costo reportado es el costo de acceso, es decir, la posicion i del elemento solicitado antes de aplicar el movimiento al frente. Las trazas completas por solicitud se obtienen ejecutando:

```
uv run python -m src.main --project-activities
```

Actividad 1

Secuencia:

$$[0, 1, 2, 3, 4, 0, 1, 2, 3, 4, 0, 1, 2, 3, 4, 0, 1, 2, 3, 4]$$

Costo total de acceso usando MTF:

90

La primera vuelta cuesta $1 + 2 + 3 + 4 + 5 = 15$. Despues de aplicar MTF, las solicitudes vuelven a pedir elementos que quedaron al fondo, por lo que la mayoria de accesos siguientes cuestan 5.

Actividad 2

Secuencia:

[4, 3, 2, 1, 0, 1, 2, 3, 4, 3, 2, 1, 0, 1, 2, 3, 4]

Costo total de acceso usando MTF:

67

Actividad 3

Una secuencia de 20 solicitudes que obtiene el minimo costo total para la configuracion inicial [0, 1, 2, 3, 4] es:

[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]

El costo total es:

20

Esto es minimo porque cada solicitud cuesta al menos 1, y pedir siempre el elemento que ya esta al frente alcanza ese limite inferior.

Actividad 4

Una secuencia de 20 solicitudes que obtiene el peor caso para MTF es pedir siempre el ultimo elemento de la configuracion actual. Para la lista inicial dada, una secuencia que logra esto es:

[4, 3, 2, 1, 0, 4, 3, 2, 1, 0, 4, 3, 2, 1, 0, 4, 3, 2, 1, 0]

El costo total es:

100

Esto es el peor caso porque la lista tiene 5 elementos y ningun acceso puede costar mas de 5. La secuencia anterior fuerza costo 5 en cada una de las 20 solicitudes.

Actividad 5

Para la secuencia de veinte solicitudes del elemento 2:

[2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2]

El costo total de acceso usando MTF es:

22

Para la secuencia de veinte solicitudes del elemento 3:

[3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3]

El costo total de acceso usando MTF es:

$$23$$

El patron observado es que, si se repite 20 veces el mismo elemento, el primer acceso cuesta la posicion inicial del elemento y los otros 19 accesos cuestan 1, porque MTF mueve el elemento al frente despues de la primera solicitud. Si la posicion inicial es p , entonces:

$$\text{costo} = p + 19$$

Actividad 6

IMTF usa mirada hacia adelante. Si el elemento accedido esta en la posicion i , se mueve al frente solo si vuelve a aparecer dentro de las proximas $i - 1$ solicitudes.

Para la secuencia de minimo costo de MTF, IMTF obtiene:

$$20$$

Para la secuencia de peor caso de MTF, IMTF obtiene:

$$60$$

En el peor caso de MTF, IMTF mejora el resultado porque evita mover al frente elementos que no aparecen suficientemente pronto en la mirada hacia adelante. Por eso conserva la configuracion original y procesa cada bloque $[4, 3, 2, 1, 0]$ con costo $5 + 4 + 3 + 2 + 1 = 15$, repetido cuatro veces:

$$4 \cdot 15 = 60$$

Experimentos

El diseno del proyecto contempla varios tipos de secuencias: accesos repetidos, secuencias adversariales, accesos aleatorios y operaciones con insercion/eliminacion. El informe final se concentra en dos experimentos representativos de acceso porque son los que permiten comparar tambien contra **OfflineOptimalReference**, cuya implementacion exacta esta limitada a secuencias pequenas de solo accesos.

Esta decision mantiene alineado el experimento con el objetivo central del material: comparar algoritmos on-line contra una referencia offline y observar el papel del potencial.

Experimento 1: accesos repetidos

Lista inicial:

$$[A, B, C, D]$$

Secuencia:

$$\sigma = [D, D, D, D]$$

Resultado:

Algoritmo	Costo total	Razon contra offline	Estado final
StaticList	16	2.29	[A, B, C, D]
MoveToFront	10	1.43	[D, A, B, C]
Bit	10	1.43	[D, A, B, C]
OfflineOptimalReference	7	1.00	[D, A, B, C]

Analisis de potencial para **MoveToFront** contra la referencia offline:

Paso	Elemento	Costo real	Φ	$\Delta\Phi$	Costo amortizado
1	D	7	0	0	7
2	D	1	0	0	1
3	D	1	0	0	1
4	D	1	0	0	1

Interpretacion:

MoveToFront aprovecha la localidad de referencia. Paga caro la primera vez porque D inicia en la cuarta posicion, pero despues D queda al frente y los siguientes accesos cuestan 1.

La referencia offline logra costo 7 porque conoce el futuro y puede reorganizar antes del primer acceso. En este caso, despues del primer paso, **MoveToFront** y la referencia offline quedan alineados, por eso $\Phi = 0$.

Experimento 2: secuencia adversarial

Lista inicial:

$$[A, B, C, D]$$

Secuencia:

$$\sigma = [D, C, B, A]$$

Resultado:

Algoritmo	Costo total	Razon contra offline	Estado final
StaticList	10	1.00	[A, B, C, D]
MoveToFront	28	2.80	[A, B, C, D]
Bit	21	2.10	[A, B, D, C]
OfflineOptimalReference	10	1.00	[A, B, C, D]

Analisis de potencial para **MoveToFront** contra la referencia offline:

Paso	Elemento	Costo real	Φ	$\Delta\Phi$	Costo amortizado
1	D	7	3	3	10
2	C	7	4	1	8
3	B	7	3	-1	6
4	A	7	0	-3	4

Interpretacion:

Esta secuencia castiga a **MoveToFront**. Cada acceso pide un elemento que esta al fondo de la lista actual, por lo que el algoritmo paga 7 en cada paso.

StaticList y la referencia offline cuestan 10 porque el orden original ya favorece la secuencia inversa en terminos de costo total de acceso sin reorganizacion pagada. Este caso muestra por que el analisis competitivo considera adversarios.

Verificacion

La suite de pruebas se ejecuto con:

```
uv run pytest
```

Resultado:

```
24 passed
```

Tambien se verifico la ejecucion del CLI con:

```
uv run python -m src.main data/examples/repeated_accesses.json --bit-seed 1 --include-offline --referen
uv run python -m src.main data/examples/adversarial_sequence.json --bit-seed 1 --include-offline --refe
uv run python -m src.main --project-activities
```

Conclusiones

MoveToFront funciona bien cuando existe localidad de referencia. El experimento de accesos repetidos lo muestra claramente: despues del primer acceso, el costo cae a 1 por solicitud.

El mismo algoritmo puede comportarse mal ante secuencias adversariales. En la secuencia $[D, C, B, A]$, cada movimiento al frente perjudica el siguiente acceso y el costo total sube a 28.

Bit reduce movimientos respecto de **MoveToFront**, por lo que puede mejorar en algunas secuencias adversariales, aunque depende de los bits iniciales y de la secuencia.

La referencia offline no representa un algoritmo on-line realista, sino una herramienta de comparacion. Su valor esta en mostrar cuanto se podria mejorar si se conociera el futuro.

El metodo del potencial permite observar como las decisiones de reorganizacion cambian el estado relativo entre dos listas. Por eso es una herramienta natural para conectar analisis amortizado con analisis competitivo.

Como generar el PDF

El PDF se genera con **pandoc**:

```
pandoc docs/informe.md -o docs/informe.pdf --pdf-engine=xelatex
```

Si **xelatex** no esta disponible, se puede usar otro motor PDF instalado en el entorno.

Apendice: reproducibilidad

Para reproducir el proyecto desde cero:

```
uv sync --dev
uv run pytest
uv run python -m src.main data/examples/repeated_accesses.json --bit-seed 1 --include-offline --referen
uv run python -m src.main data/examples/adversarial_sequence.json --bit-seed 1 --include-offline --refe
```

```
uv run python -m src.main --project-activities  
pandoc docs/informe.md -o docs/informe.pdf --pdf-engine=xelatex
```

Los resultados completos de las ejecuciones usadas en el informe estan respaldados en:

- docs/resultados/accesos-repetidos.md
- docs/resultados/secuencia-adversarial.md
- docs/resultados/actividades-proyecto3.md

Los datos del estudiante fueron incluidos en el encabezado del informe.